

## Exercise 1: Basic Spring Application (Library Management System)

### Scenario

This exercise demonstrates how to create a basic Spring application using XML-based configuration. The application manages a simple Library system using BookService and BookRepository classes.

---

### Objective

The objective of this exercise is to understand how to:

- Configure Spring application using XML
  - Create and manage beans using Spring IoC container
  - Load Spring application context
  - Understand basic Spring application flow
- 

### 1. Understanding Spring Core

Spring Core provides the IoC (Inversion of Control) container which is responsible for:

- Creating objects (beans)
- Managing dependencies
- Controlling application flow

Instead of manually creating objects using new, Spring manages them automatically.

---

### 2. Configuration (applicationContext.xml)

```
<bean id="bookRepository" class="com.library.repository.BookRepository"/> <bean  
id="bookService" class="com.library.service.BookService"/>
```

---

### 3. Implementation

BookRepository.java

```
public class BookRepository {  
  
    public void saveBook() {  
        System.out.println("Book saved in repository");  
    }  
  
}
```

---

BookService.java

```
public class BookService {  
  
    public void addBook() {  
        System.out.println("Service: Adding book...");  
    }  
  
}
```

---

Main Application

```
ApplicationContext context =  
new ClassPathXmlApplicationContext("applicationContext.xml");  
  
BookService service =  
(BookService) context.getBean("bookService");  
  
service.addBook();
```

---

#### 4. Output

```
Service: Adding book...  
Book saved in repository
```

---

#### 5. Conclusion

This exercise successfully demonstrates the creation of a basic Spring application using XML configuration and Spring IoC container.

---

### Exercise 2: Dependency Injection in Spring

#### Scenario

This exercise demonstrates how Dependency Injection (DI) is used in Spring to connect BookService and BookRepository using XML configuration.

---

#### Objective

The objective of this exercise is to understand how to:

- Implement Dependency Injection using Spring
  - Use setter injection in Spring
  - Configure dependencies using XML
  - Understand IoC container behavior
-

## 1. Understanding Dependency Injection

Dependency Injection is a design pattern where Spring automatically injects required dependencies instead of manually creating them.

---

## 2. Configuration (applicationContext.xml)

```
<bean id="bookRepository" class="com.library.repository.BookRepository"/> <bean  
id="bookService" class="com.library.service.BookService"> <property name="bookRepository"  
ref="bookRepository"/> </bean>
```

---

## 3. Implementation

BookRepository.java

```
public class BookRepository {  
  
    public void saveBook() {  
        System.out.println("Book saved in repository");  
    }  
  
}
```

---

BookService.java

```
public class BookService {  
  
    private BookRepository bookRepository;  
  
    public void setBookRepository(BookRepository bookRepository) {  
        this.bookRepository = bookRepository;  
    }  
  
    public void addBook() {  
        System.out.println("Service: Adding book...");  
        bookRepository.saveBook();  
    }  
  
}
```

---

## 4. Output

```
Service: Adding book...  
Book saved in repository
```

---

## 5. Conclusion

This exercise successfully demonstrates Dependency Injection using Spring XML configuration and setter-based injection.

---

## **Exercise 4: Maven Project Configuration (Library Management System)**

### **Scenario**

This exercise demonstrates how to configure a Maven project for a Spring-based application by adding required dependencies and build plugins.

---

### **Objective**

The objective of this exercise is to:

- Configure Maven project structure
  - Add Spring dependencies
  - Set Java compiler version
  - Configure Maven build plugins
- 

### **1. Understanding Maven Configuration**

Maven is a build tool used to:

- Manage project dependencies
  - Automate build process
  - Ensure consistent project structure
- 

### **2. Dependencies (pom.xml)**

```
<dependency> <groupId>org.springframework</groupId> <artifactId>spring-context</artifactId>  
<version>5.3.30</version> </dependency>
```

---

### **3. Maven Compiler Plugin Configuration**

```
<plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-  
plugin</artifactId> <version>3.11.0</version> <configuration> <source>1.8</source>  
<target>1.8</target> </configuration> </plugin>
```

---

### **4. Result**

The Maven project was successfully configured with Spring dependencies and Java 1.8 compiler support.

## 5. Conclusion

This exercise successfully demonstrates Maven configuration for a Spring-based application, including dependency management and build setup.

# OUTPUT SCREENSHOT

